

TRABALHO ACADÊMICO

TESTES UNITÁRIOS EM PYTHON: UNITTEST E PYTEST

Comparação entre duas bibliotecas/frameworks de testes unitários

1. Introdução

Os testes unitários são uma prática fundamental no desenvolvimento de software, pois têm como objetivo verificar se cada componente individual de um sistema funciona corretamente. A utilização de testes automatizados ajuda a identificar erros com maior rapidez, reduzir problemas durante a manutenção e aumentar a confiabilidade do software.

Existem diversas bibliotecas e frameworks destinados à realização de testes unitários em diferentes linguagens de programação. Neste trabalho são analisadas duas ferramentas da linguagem Python: Unittest e Pytest.

2. Bibliotecas escolhidas

As bibliotecas escolhidas foram Unittest e Pytest. Ambas permitem criar e executar testes automatizados em Python, porém apresentam diferenças importantes em relação à sintaxe, organização dos testes e apresentação dos resultados.

3. Pontos positivos e negativos

Característica	Unittest	Pytest
Linguagem	Python	Python
Facilidade de uso	Média	Alta
Necessita classes	Geralmente sim	Não
Sintaxe	Mais extensa	Mais simples
Assertions	<code>self.assertEqual()</code> , <code>self.assertTrue()</code>	<code>assert</code> do Python
Instalação	Já vem com Python	Precisa ser instalado
Plugins	Possui integrações	Grande quantidade de plugins
Organização	Estruturada	Flexível

3.1 Unittest — pontos positivos

- Faz parte da biblioteca padrão do Python, não sendo necessária uma instalação adicional.
- Possui uma estrutura bem definida para organização dos testes.
- É bastante utilizado em projetos profissionais e acadêmicos.
- Possui recursos como `setUp()` e `tearDown()`.
- Disponibiliza métodos como `assertEqual()`, `assertTrue()` e `assertRaises()`.

3.2 Unittest — pontos negativos

- Sua sintaxe pode ser considerada mais extensa.
- Normalmente utiliza classes que herdam de `unittest.TestCase`.
- Para testes simples, pode exigir mais código do que o necessário.

- A leitura pode ser menos intuitiva para iniciantes.

3.3 Pytest — pontos positivos

- Possui sintaxe simples e fácil de compreender.
- Permite escrever testes sem a necessidade de criar classes.
- Utiliza o assert padrão do Python.
- Possui recursos avançados, como fixtures e parametrização.
- É adequado para projetos pequenos e grandes.
- Apresenta mensagens de erro detalhadas.

3.4 Pytest — pontos negativos

- Não faz parte da biblioteca padrão do Python, sendo necessário instalá-lo.
- A grande quantidade de recursos e plugins pode aumentar a curva de aprendizado.
- Projetos que seguem especificamente o padrão unittest podem exigir adaptação.

4. Exemplo de utilização

Para demonstrar as diferenças entre as duas ferramentas, será utilizada uma função simples de soma.

Arquivo calculadora.py

```
def somar(a, b):  
    return a + b
```

4.1 Utilizando unittest

```
import unittest  
from calculadora import somar  
  
class TestCalculadora(unittest.TestCase):  
  
    def test_somar(self):  
        resultado = somar(2, 3)  
        self.assertEqual(resultado, 5)  
  
if __name__ == "__main__":  
    unittest.main()
```

O unittest utiliza uma classe que herda de `unittest.TestCase`. O método `self.assertEqual()` verifica se o resultado obtido é igual ao valor esperado.

Execução:

```
.  
-----  
Ran 1 test in 0.001s  
  
OK
```

4.2 Utilizando Pytest

Primeiramente, é necessário instalar a biblioteca utilizando o comando:

```
pip install pytest

from calculadora import somar

def test_somar():
    resultado = somar(2, 3)
    assert resultado == 5
```

No Pytest não é necessário criar uma classe. O teste pode ser escrito diretamente como uma função e utiliza o assert nativo do Python.

Execução:

```
===== test session starts =====
collected 1 item

test_calculadora_pytest.py . [100%]

===== 1 passed in 0.05s =====
```

5. Diferenças de implementação e resposta

A principal diferença entre as duas ferramentas está na forma de implementação. O Unittest utiliza uma estrutura baseada em classes e métodos específicos da biblioteca, enquanto o Pytest permite utilizar funções simples e o assert nativo do Python.

Também existem diferenças na saída apresentada ao usuário. O Unittest apresenta uma saída mais tradicional, indicando o número de testes executados e se o resultado foi OK. O Pytest apresenta uma saída mais detalhada, incluindo a quantidade de testes coletados, a porcentagem de execução e informações detalhadas quando ocorre uma falha.

6. Conclusão

O Unittest é uma boa opção quando se deseja utilizar uma ferramenta incluída no Python e que apresenta uma estrutura padronizada para criação dos testes. Sua principal desvantagem é possuir uma implementação mais extensa e, em alguns casos, menos intuitiva.

O Pytest apresenta uma sintaxe mais simples e permite criar testes com poucas linhas de código. Além disso, possui recursos avançados e uma grande quantidade de plugins, sendo uma opção bastante flexível. Sua principal desvantagem é a necessidade de instalação adicional.

Dessa forma, ambas as ferramentas são capazes de realizar testes unitários de maneira eficiente. Para projetos que valorizam simplicidade e rapidez na criação dos testes, o Pytest pode ser uma escolha mais conveniente. Já para projetos que precisam seguir uma estrutura tradicional e padronizada de testes, o Unittest pode ser mais adequado.

7. Referências

PYTHON SOFTWARE FOUNDATION. unittest — Unit testing framework. Python Documentation.

PYTEST DEVELOPMENT TEAM. pytest documentation. Documentation for pytest.